robot 仓库新手指南 v2（给 Justin）

> 写给仓库主人 Justin（自称新手）。
> 上一版太难：这一版把 **8 件事各自写成一章**。每章都有：生活类比 → 图 → 具体数字 →「这一步你要做什么」清单。
> 命令、路径都对照过本机克隆 `/workspace/robot`（提交 `c522807`，含 main 上已有文件与当前工作区里的 iPhone/双目文档）。
> 适合提交为仓库内 `docs/BEGINNER_GUIDE.md`。同目录另有离线 `guide.html` / `guide.pdf`。
> 日期：2026-10-11（Asia/Shanghai）。

目录

0. 先看这一页：整件事像做饭

1. 要采什么数据（文件夹长什么样）

2. 每种数据是什么意思

3. 跟一帧画面走完全程

4. WiLoR 怎么认出双手

5. 为什么要有 IMU 和时间戳

6. 相机位姿与 SLAM/VIO：手从「镜头前」变成「房间里」

7. 训练数据长什么样（BC / ACT）

8. 常见翻车：用人话讲

9. 附录：命令速查与名词

---

0. 先看这一页：整件事像做饭

一句话：头上（或 iPhone）录自己的手 → 认出手指在哪、离镜头多远 → 扔掉坏片段 → 打包成教材 → 教模型「下一小步手怎么动」。

做饭类比	仓库里叫什么
拍你切菜的视频	采集（双目头戴 / iPhone 激光雷达 / 公开数据）
标出每根手指	WiLoR 认手
量菜离镜头几厘米	双目三角化 或 激光雷达深度
剪掉糊掉、手出画的镜头	QC（质检）
做成统一课件格式	LeRobot 数据集
学生照着学下一刀怎么切	行为克隆 BC / ACT

```
原料(session/) → 认手 → 量深度 → 世界坐标 → 质检 → LeRobot → 训练
```

软件大多已能跑；双目头戴还在等硬件；临时可用 **iPhone Pro + Record3D**。小模型常打不过「手永远不动」——多半是数据太少，不是代码全坏了。

---

1. 要采什么数据（文件夹长什么样）

1.1 类比

像拍烹饪短视频，还要附赠：

- **彩色画面**：观众能看懂你在干什么
- **两只眼睛（双目）或激光尺（LiDAR）**：知道手离镜头多远（厘米）
- **手机陀螺仪（IMU）**：头怎么晃
- **每帧几点几分（时间戳）**：多路数据能对上
- **尺子标定（calib）**：像素 ↔ 真实米
- **可选配音/文字**：这道菜叫什么（语言标注）

1.2 两条采集路（选一条即可）

路线	适合谁	深度怎么来	相机在房间里的位置
A. 双目头戴	正式设备（方案 A 约 727–982 元，见 `docs/headcam_stereo_bom.md`）	左右目三角化	自己跑 SLAM/VIO → `slam.tum`
B. iPhone Pro	双目到货前临时采	背面激光雷达	Record3D 导出的 ARKit → `slam.tum`

普通 iPhone（无激光雷达）**不能**走 B。

1.3 参数要求（请对表检查）

双目头戴（规格 + Day1 清单）

项	要求	人话

帧率	**30 fps**（或 60）	每秒至少 30 张；太慢手指会糊		
分辨率	≥640×400 够用；彩色参考常≥720p	手要有足够像素		
基线	**8–10 cm**（至少 6 cm；校验允许 4–15 cm）	两只「眼睛」间距；写错成毫米会灾难		
视场	约 **100°**（别选 75°）	太窄则手常只进一目		
左右同步	`	左曝光–右曝光	` **≤ 1 ms**	不同时眨眼，深度会飘
IMU	建议 ≥100 Hz；与相机刚性固定	给 SLAM/VIO 用		
深度单位	**米**（不是毫米）	手腕常见 0.2–0.8 m		

iPhone（`docs/iphone_capture.md`）

项	要求
机型	12 Pro 及以后 Pro / Pro Max
App	Record3D；后置 **LiDAR**，不要前置 FaceID
帧率	30 fps（60 fps 可用 `--stride 2`）
手距	**≥ 25 cm**，最好 30–70 cm；量程约 0.25–5 m
深度文件	`depth.npz`：米制 float16 + 置信度 0/1/2

1.4 文件夹长什么样（本仓库真实约定）

双目会话（`docs/headcam_data_spec.md` §7.7）：

```
session/
  metadata.json              # episode_id、fps、task、宽高…
  timestamps.csv             # frame_index,timestamp_s
  imu.csv                    # timestamp_s,gx,gy,gz,ax,ay,az
  calib.yaml                 # 双目内参/外参
```

```
  rgb.mp4                  # 可选彩色；没有则用左目
  slam.tum                 # 可选；相机世界位姿
  stereo/
    left.mp4
    right.mp4
    timestamps_lr.csv      # frame_index,left_s,right_s
```

iPhone 会话（`docs/iphone_capture.md` §6，适配器输出）：

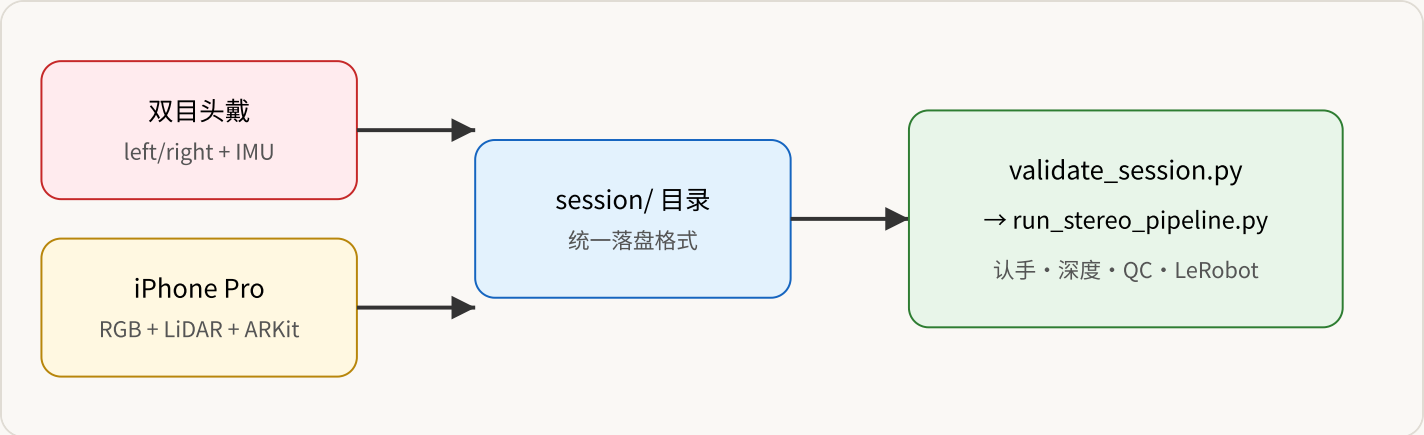
```
session/
  rgb.mp4
  depth.npz                # depth (N,dh,dw) 米; conf 0/1/2
  calib.yaml               # sensor: iphone_lidar, K_rgb…
  slam.tum                 # ARKit，已转到 OpenCV 约定
  timestamps.csv           # frame,timestamp_s
  metadata.json
```

假录制器（设备未到时练手）：

```
python3 scripts/simulate_device_session.py \
  --hot3d data/train_quest3/clip-000000 \
  --out outputs/fake_session \
  --synth-imu
```

`--synth-imu` 从位姿反推 IMU，**只测格式**，不是真噪声。

1.5 图：两路数据进同一条工厂



1.6 具体数字例子

假设你录 **30 秒、30 fps**：

- 约 **900 帧**彩色（或左右各 900）
- `timestamps.csv` 约 900 行；相邻差约 **0.0333 s**
- IMU 若 200 Hz：约 **6000 行** `imu.csv`
- 基线若尺子量 **9.0 cm**，`calib.yaml` 里也应约 **0.09 m**（差 ⩽ 2 mm）
- iPhone：手腕深度多数应在 **0.25–1.5 m**（管线检查）

1.7 这一步你要做什么

- [ ] 选定路线 A（双目）或 B（iPhone Pro）
- [ ] 按 §1.3 表核对 fps / 分辨率 / 基线 / 同步 / 深度单位
- [ ] 落盘成上面的 `session/` 结构（或用 Record3D → 适配器）
- [ ] 跑：`python3 scripts/validate_session.py /path/to/session`
- [ ] 必须看到「通过」再进下一步；Day1 细节见 `docs/hardware_day1_checklist.md`
- [ ] iPhone：先读 `docs/iphone_capture.md`；双目总图见 `docs/PIPELINE_ARCHITECTURE.md`

---

2. 每种数据是什么意思

2.1 类比

数据类型	像什么
彩色帧 RGB	一张菜照
深度 depth	激光尺：每个像素「离镜头几米」
IMU 加速度/陀螺	头上绑了运动手环
相机位姿	「摄影师站在厨房哪、镜头朝哪」
手部 21 关节	手指骨架上的 21 个图钉
动作增量 action	「下一刀相对现在挪了多少」

2.2 图：一帧里有什么



2.3 真实字段（来自 `scripts/egodata/schema.py` + 导出约定）

统一 episode：`schema_version = "1.0"`；21 点顺序与 MediaPipe Hands 相同：

`wrist, thumb_cmc, thumb_mcp, thumb_ip, thumb_tip, index_mcp… pinky_tip`（共 21）。

片段 JSON 骨架（示意，数字为示例）：

```
{
  "schema_version": "1.0",
  "episode_id": "sess01_pick_cup",
  "fps": 30.0,
  "coordinate_frame": "slam_world",
  "num_frames": 3,
  "timestamps": [0.0, 0.0333, 0.0667],
  "camera_poses": [
    [[1,0,0,0],[0,1,0,0],[0,0,1,0],[0,0,0,1]]
  ],
  "hands": {
    "right": {
      "joints": [
        [[0.12, -0.05, 0.48], [0.11, -0.04, 0.47], null]
      ],
      "wrist_pose": [[0.12, -0.05, 0.48, 0.0, 0.0, 0.0, 1.0]],
      "confidence": [0.92]
    },
    "left": {
      "joints": [[[null, null, null]]],
      "wrist_pose": [[null, null, null, null, null, null, null]],
      "confidence": [0.1]
    }
  }
}
```

说明：

- `joints[帧][0..20]`：每个点 `[x,y,z]`，单位**米**；缺测写 `null`，**禁止用 0 冒充**
- `wrist_pose`：7 个数 = 位置 xyz + 四元数 xyzw
- `confidence`：0～1；导出时数字 **< 0.5** 常当无效

IMU CSV（假录制器表头，`scripts/simulate_device_session.py`）：

```
timestamp_s,gx,gy,gz,ax,ay,az  # synthesized_from_poses
0.000000000,0.01,-0.02,0.00,0.05,0.10,9.75
0.005000000,0.01,-0.01,0.01,0.06,0.09,9.74
```

- `gx,gy,gz`：角速度（绕三轴转多快）
- `ax,ay,az`：比力（含重力感觉）

- 转换进 episode 时**只记路径和行数**，不在 JSON 里积分（规格 §7.7）

时间戳 CSV：

```
frame_index,timestamp_s
0,0.000000000
1,0.033333333
2,0.066666667
```

左右同步另有：`stereo/timestamps_lr.csv` → `frame_index,left_s,right_s`。

动作增量（训练用，不是原始录制字段）

导出后每行 `action` 长度默认 **14**：

- 左手：`dxyz` (3) + 相对四元数(4)
- 右手：同样 7 维
- 「不动」= `dxyz=0` 且四元数 `0,0,0,1`

见 `scripts/egodata/lerobot_export.py`。

仓库里有效手标记叫 **`observation.hand_valid`**（左右各 0/1），不是 `action_valid`。含义一样：这只手这帧能不能当监督信号。

2.4 具体数字例子

第 42 帧、右手抓杯：

字段	例子	人话
彩色	1920×1080 一帧	照片
手腕相机系	`(0.12, -0.05, 0.48)` m	大约半米远
手掌长	~9 cm	手腕到中指根
置信度	`0.92`	挺有把握
action（下一步）	左手不动 + 右手 `dxyz≈(0.01,0,0)`	右手往右挪 1 cm

2.5 这一步你要做什么

- [ ] 打开任意通过管线的 `episodes/*.json`，找到 `hands.right.joints[0]`
- [ ] 确认单位是米：手腕 z 多在 **0.2～0.8**，不是 200～800
- [ ] 对照 `scripts/egodata/schema.py` 里的 `MEDIAPIPE_21` 点名
- [ ] 看一眼 `imu.csv` / `timestamps.csv` 表头是否如上
- [ ] 记住：训练看的是 **下一步增量**，不是绝对坐标本身

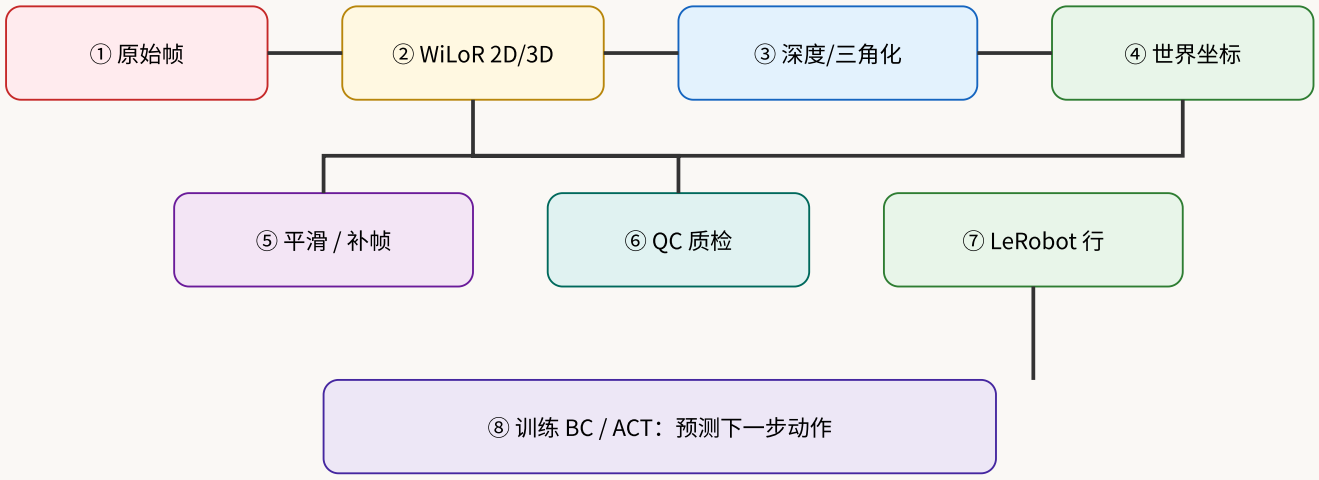
---

3. 跟一帧画面走完全程

3.1 类比

你把杯子从桌上拿起。我们只盯 **第 42 帧**，看它怎么变成训练里的一行。

3.2 图：一帧的流水线



入口：scripts/run_stereo_pipeline.py

（双目 / HOT3D / iPhone 同一条命令）

红=原始 · 黄=认手 · 蓝=深度 · 绿=可用训练数据

3.3 逐步发生了什么（带数字）

1. **原始**：左目（或 RGB）里右手在画面中下；`t=1.400 s`（若 30 fps，约第 42 帧）。

2. **WiLoR**：检出框 → MANO 参数 → **21 个 2D 点 + 单目 3D**（单目尺度常不准）。代码：`scripts/headcam/hand_pose.py`。

3. **深度**：

- 双目：左右 2D + `calib.yaml` → 三角化到左相机系（米）—— `scripts/headcam/stereo_pipeline.py`

- iPhone：手区域取 LiDAR 深度再对齐手型—— `scripts/headcam/rgbd_pipeline.py`

例：手腕相机系 `(0.12,-0.05,0.48)` m。

4. **世界坐标**：乘 `T_world_cam`（来自 `slam.tum`）。没有轨迹时仍是相机系，`coordinate_frame` 写 `camera`。

5. **平滑**：默认世界系手腕 RTS；可补 ≤5 帧（标 `filled`，置信度 0）。

6. **QC**：手出画、糊、干等…坏帧比例 **> 20%** 整段拒绝（`scripts/egodata/qc.py`）。

7. **LeRobot**：通过的段写成 parquet + mp4；`observation.state` **140** 维，`action` **14** 维下一步增量。

8. **训练**：线性 BC 或 ACT 学「看到状态/图像 → 下一步怎么动」。

3.4 一条命令（已核对 `--help`）

```
cd /workspace/robot
# 双目会话
python3 scripts/validate_session.py /data/sess01
python3 scripts/run_stereo_pipeline.py --session /data/sess01 --out outputs/sess01

# HOT3D 公开双目
python3 scripts/run_stereo_pipeline.py \
  --hot3d data/train_quest3/clip-000000 --out outputs/hot3d

# iPhone
source /workspace/stereo_env.sh   # 本机 WiLoR/MANO 环境变量示例
python3 scripts/run_stereo_pipeline.py \
  --iphone ~/iphone_raw/take1.r3d --out outputs/iphone
```

输出常见：`episodes/`、`qc/yield.html`、`lerobot/`、`report.md`。

3.5 这一步你要做什么

- [ ] 任选一条命令跑通（没有真数据可用 HOT3D 或假会话）
- [ ] 打开 `outputs/.../report.md`，看产出率与拒绝原因
- [ ] 打开一条 `episodes/*.json`，核对第 0 帧手腕是否在米级
- [ ] 确认 `lerobot/` 下有 `meta/` 与 `data/`（若未加 `--no-export`）
- [ ] 架构总表：`docs/PIPELINE_ARCHITECTURE.md`

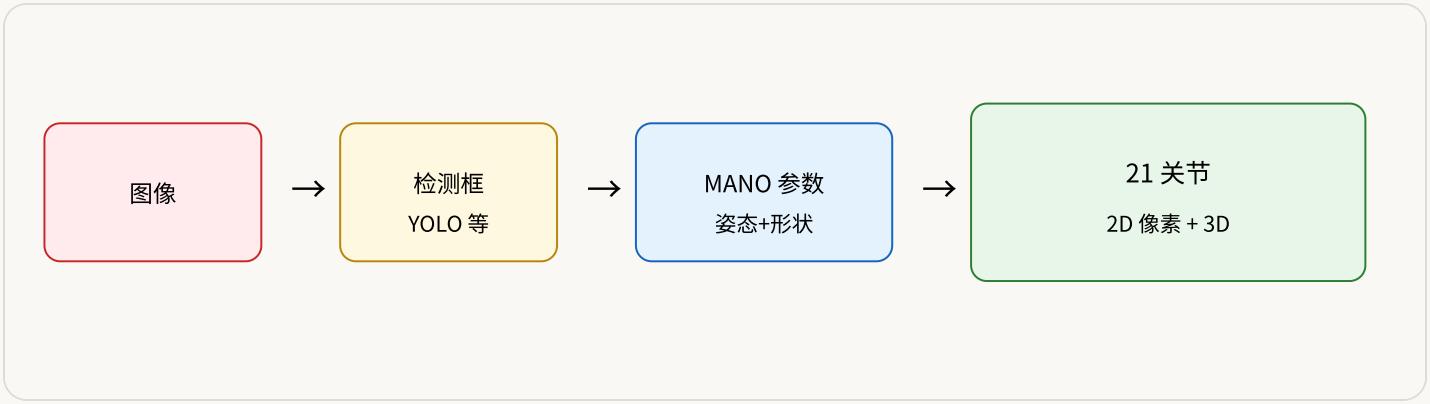
4. WiLoR 怎么认出双手

4.1 类比

像请一位「看照片画手骨」的专家：

1. 先在照片上圈出「哪有手」（检测框）
2. 再往标准手模 **MANO** 上套参数（弯曲、长短）
3. 吐出 **21 个关节点**

4.2 图：输入 → 21 点



4.3 左右手是什么意思

- JSON / 训练里的 `left` / `right` = **穿戴者自己的左手/右手**（第一人称）。
- WiLoR/HaMeR 常用 **右手 MANO**，左手由镜像得到（见 `hand_pose.py` 文档字符串）。
- 画面里「看起来在左边的手」不一定是左手——头转了会反。管线有关联/轨迹修正（`--no-assoc` 可关）。

4.4 为什么必须有双目或 LiDAR（修尺度）

单目像「不知远近的平面照片」：手可以是「小手很近」或「大手很远」。

条件	手腕误差量级（仓库已有数）
EgoDex 上 WiLoR 单目（未对齐）	约 **10–22 cm**
HOT3D 双目三角化	中位约 **1.22 cm**

iPhone LiDAR 代理（DexYCB）	中位约 **1.41–1.51 cm**

结论：认手形状靠 WiLoR；**米制远近**靠双目或激光雷达。

代码位置：`scripts/headcam/hand_pose.py`

（三角化：`stereo_pipeline.py`；RGB-D：`rgbd_pipeline.py`）

环境变量示例（本机 `/workspace/stereo_env.sh`）：

```
export MANO_MODEL_DIR=/workspace/mano
export WILOR_CHECKPOINT=/workspace/WiLoR/pretrained_models/wilor_final.ckpt
export WILOR_DETECTOR=/workspace/WiLoR/pretrained_models/detector.pt
export WILOR_CONFIG=/workspace/WiLoR/pretrained_models/model_config.yaml
```

MANO 权重有许可限制，**不进仓库**；从 https://mano.is.tue.mpg.de 自行下载。

4.5 具体数字例子

- 检测框：例如像素矩形 `(320,400)-(900,900)`
- 21 点 2D：手腕约 `(610, 720)` px
- 单目 3D：可能把手腕放到错误深度（数米外）
- 三角化/LiDAR 后：拉回约 **0.48 m**
- 置信度：`0.9`；补帧 `filled` 时置信度写成 **0**

4.6 这一步你要做什么

- [ ] 确认读过 `scripts/headcam/hand_pose.py` 顶部说明（MANO / 虚拟焦距）
- [ ] GPU 上优先 `--backend wilor`；没 GPU 可 `mediapipe`（差很多）
- [ ] Colab：`notebooks/headcam_hand_pose_colab.ipynb`
- [ ] 看缓存：`outputs/.../cache/*.wilor.json`（改参数重跑更快）
- [ ] 不要指望单目绝对深度；务必接双目或 iPhone LiDAR

---

5. 为什么要有 IMU 和时间戳

5.1 类比

拍烹饪视频时：

- **时间戳** = 每张照片上的时钟；没有它，旁白、特写对不齐
- **IMU** = 头上的运动传感器：转头、点头有多快

三者分工：

东西	干什么	不干什么
时间戳	对齐视频、深度、IMU、SLAM	不告诉你手在哪
IMU	短期旋转、辅助 VIO	**不能**直接当手部姿态
相机画面	认手、纹理	单独不够世界坐标

5.2 图：多路对齐



5.3 具体用途（对照 `validate_session.py`）

1. **同步左右相机**：读 `stereo/timestamps_lr.csv`，默认 `sync_ms.max ≤ 1`。差 10 ms 时，HOT3D 仿真里「手腕误差 ≤2 cm」比例会变差。

2. **估计头/相机运动**：IMU + 视觉 → VIO/SLAM → `slam.tum`。

3. **发现丢帧**：时间戳应严格递增；间隔相对中位数抖 **>50%** 会记 jitter；`dropped_or_jitter_frames` 应接近 0。

4. **帧率核对**：由时间戳推算的 fps 要与 `metadata.json` 的 fps 一致（相对误差默认 5% 内）。

5.4 具体数字例子

- 视频 30 fps → 帧间隔 **33.3 ms**
- 左右差 **0.3 ms** → 合格；**5 ms** → `validate_session` 可注入故障 `desync_5ms` 测抓取
- IMU 200 Hz → 每 5 ms 一行；应覆盖 `[第一帧，最后一帧]`
- 缺 `timestamps.csv`：只能按 `frame/fps` 瞎猜，**无法和 SLAM/IMU 严面对齐**（会 WARN）

5.5 这一步你要做什么

- [ ] 录制程序写 `timestamps.csv`；双目再写 `timestamps_lr.csv`
- [ ] 有设备时写真实 `imu.csv`；HOT3D 没有 IMU 属正常
- [ ] 跑 `python3 scripts/validate_session.py SESSION --json report.json`，看 `sync_ms`、`measured_fps`、`imu_hz`
- [ ] 对着闪烁 LED 或手机毫秒秒表，肉眼核对左右是否同闪
- [ ] 记住：IMU 测的是**头**，不是手

---

6. 相机位姿与 SLAM/VIO：手从「镜头前」变成「房间里」

6.1 类比

- **相机系**：以镜头为原点——「手在镜头前 48 cm」
- **世界系**：以房间为原点——「手在餐桌上方」

你转头时，手相对镜头在动，但相对桌子可能没动。训练若不管这个，模型会学乱。

公式（规格 §7.7）：

```
p_world = T_world_cam · p_cam
```

`T_world_cam` 来自 `slam.tum` 里时间最近的一帧。

6.2 图：同一只手两个坐标系



6.3 SLAM / VIO 是什么

词	人话
SLAM	一边走一边画地图，并知道自己在哪
VIO	视觉 + IMU 一起估位姿（更抗抖）
TUM 文件	每行：`timestamp tx ty tz qx qy qz qw`

仓库**不内置** ORB-SLAM3 / OpenVINS；你在外部算好，只放入 `slam.tum`。

6.4 何时 iPhone ARKit 就够

双目未到货时：

- Record3D 导出位姿 → 适配器写成 `slam.tum`（OpenGL→OpenCV 约定已处理）
- **够用来**：跑通管线、做 QC、导出、小规模训练对比
- **不够当最终论文唯一精度**：视角窄、跟踪会跳；`validate_session` 用相邻帧跳变（>10 cm 或 >20°）当 WARN

正式头戴仍要自己的双目 + IMU + 标定 + SLAM。

6.5 具体数字例子

- `slam.tum` 一行：`1.400 0.02 1.10 0.80 0 0 0 1` → 相机在世界坐标约 `(0.02,1.10,0.80)` m
- 手在相机系 `(0.12,-0.05,0.48)` → 乘位姿后到桌面附近
- 闭环走一圈回原点：误差应在**厘米级**（Day1 清单）
- 无 `slam.tum`：`coordinate_frame=camera`，**还不是**规格要求的世界系标签

6.6 这一步你要做什么

- [ ] 双目：外部跑 VIO/SLAM，导出 TUM 到会话根目录 `slam.tum`
- [ ] iPhone：确认适配器已写出 `slam.tum`，且 `validate_session` 通过
- [ ] 打开 episode，看 `coordinate_frame` 是 `slam_world` / `arkit_world` 还是 `camera`
- [ ] 录制前让 ARKit/VIO 先「看房间」2–3 秒再建图
- [ ] 突然甩头导致跳变：重录；或看 report 里的 `tracking` / `jump`

---

7. 训练数据长什么样（BC / ACT）

> 关于 **XHEY**：当前克隆里**没有**名为 XHEY 的训练脚本，也没有 `mcap_adapter`。若你指某个遥操作 App 的导出，最终仍应落到本章的 **LeRobot 契约**（state / action / hand_valid / horizon）。下面按仓库里真实存在的 **线性 BC** 与 **ACT** 写。

7.1 类比

- **observation（观察）** = 现在菜和刀在哪
- **action（动作）** = 下一刀相对现在怎么挪
- **horizon** = 一次想好连续好几刀（默认 16 步）
- **train/val** = 练习题 / 摸底考（按整段 episode 留出，默认约 10%）

7.2 向量尺寸（`scripts/egodata/lerobot_export.py`）

字段	形状	含义
`observation.state`	**140**	左 21×3 + 右 21×3 + 左腕 7 + 右腕 7
`observation.hand_valid`	**2**	左/右是否可用（置信度 <0.5 或关节缺测 → 0）
`action`	**14**（默认）	下一步左右手腕增量；`--include-hand` 再加关节增量
`horizon`	**16**（默认）	写入 `meta/egodata_export.json`；parquet **每行仍是 1 步**，训练时再拼块
图像	常缩到 **224×224**	`--video-size 224`

布局：`state[0:63]` 左手关节，`[63:126]` 右手，`[126:133]` 左腕，`[133:140]` 右腕。

7.3 图：一行训练样本



7.4 怎么准备（现有脚本）

```
# 1) 统一 JSON（开放数据示例）
python3 scripts/convert_egodex.py --input test --out outputs/egodex_unified --limit 20

# 2) 质检
python3 scripts/egodata_qc.py \
  --episodes outputs/egodex_unified \
  --html outputs/yield_report.html \
  --csv outputs/yield_episodes.csv

# 3) 导出 LeRobot（只含 accepted=yes）
python3 scripts/ego_to_lerobot.py \
  --episodes outputs/egodex_unified \
  --yield-csv outputs/yield_episodes.csv \
  --out outputs/egodex_lerobot \
  --horizon 16 --video-size 224

# 4) 线性 BC（CPU 可跑；需环境有 pyarrow）
python3 scripts/ego_pretrain_bc.py \
  --dataset outputs/egodex_lerobot \
  --max-frames 1000 \
  --val-fraction 0.1 \
  --log outputs/ego_pretrain.log

# 5) ACT 多档：scripts/ego_act_scaling.py
#    或 notebooks/egodex_act_scaling_colab.ipynb
#    单档配方示例：examples/ego_act_train.yaml
```

头戴/iPhone 一条龙已含 QC+导出时，可直接对 `outputs/.../lerobot` 训练。

7.5 「ADE cm」是什么

ADE = Average Displacement Error，**平均位移误差**。

人话：把模型预测的手腕轨迹，和真轨迹比，平均偏了 **几厘米**。

- 数字越小越好
- 常和 **零动作 / 保持不动** 比：模型若 ADE ≈ 零动作 ADE，等于没学会
- 本仓库线性 BC 日志主字段是 `val_loss` 与 `copy_current_wrist`（保持不动基线）；ACT 笔记本在反归一化后的动作上算 L1。把误差说成「ADE cm」时，指的就是这种**开环轨迹平均偏差的厘米版**。

例子（旧实验口径，量级感）：零动作 ADE 约 3 cm，模型 3.06 cm → 几乎打平 → 要加数据。

7.6 具体数字例子

- 一段 30 fps、20 s → 约 600 帧；最后一帧无「下一步」，导出时丢掉 → 约 599 个 action
- `horizon=16`：需要连续 16 行才能拼一块；太短的段会被跳过
- `hand_valid=(1,0)`：只让左手动作进损失，右手当缺失
- 微调经验量级（规格附录）：有预训练时每任务约 **50–150** 段；从零约 **500–1000** 段

7.7 这一步你要做什么

- [ ] 导出后打开 `meta/info.json`，确认 `observation.state` shape `(140,)`，`action` shape `(14,)`
- [ ] 打开 `meta/egodata_export.json`，看 `horizon`、`state_doc`
- [ ] 训练时保留 `hand_valid`：缺失手不要当「保持不动」学
- [ ] 用 `copy_current_wrist` / 零动作基线对比，不要只看自己 loss 下降
- [ ] ACT：走 `ego_act_scaling.py` 或 Colab；本克隆无 `ego_act_train.py` 这个文件名

---

8. 常见翻车：用人话讲

8.1 类比

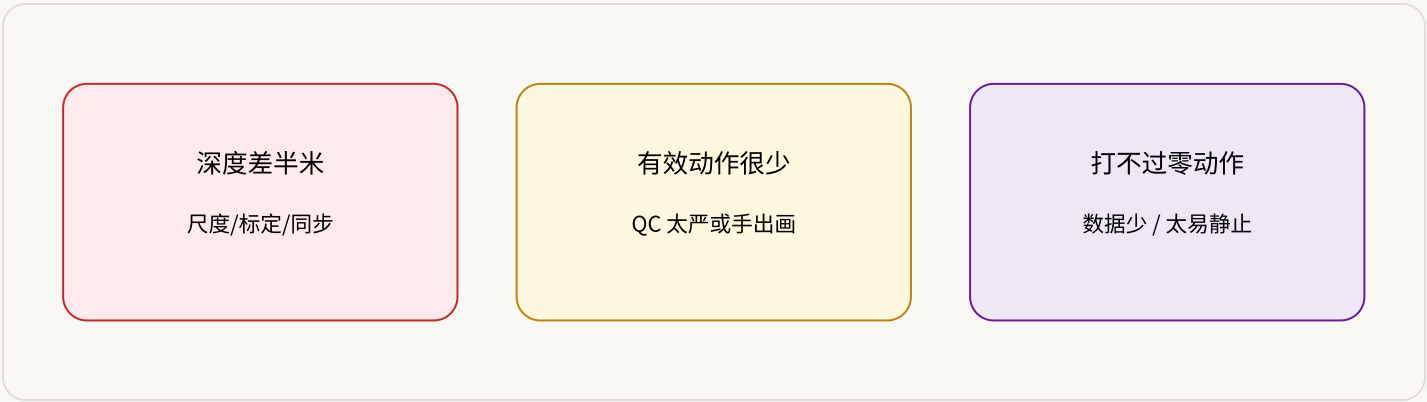
做菜翻车三种经典：

1. 盐放成糖罐——**单位错了**

2. 菜谱只有三页——**有效动作太少**

3. 学员学会「站着不动也像那么回事」——**打不过零动作**

8.2 图：三只坑



8.3 坑 1：深度差了大约半米

现象：手腕应该在 0.5 m，结果跑到 1.0 m 或 0.05 m；手掌长度系统性偏大/偏小。

常见原因：

- 基线写成毫米（0.09 写成 90）—— `validate_session` 会抓「基线不在 4–15 cm」
- 左右不同步几十毫秒
- 单目没接双目/LiDAR，尺度漂
- iPhone 用了前置深度或手太近（<25 cm）

你要做：先 `validate_session`；再看 report 里手掌是否多在 7–11 cm；用尺子做 Day1「静止尺子」测试。

8.4 坑 2：有效动作很少

现象：录了 10 分钟，QC 后只剩一点点；或 `hand_valid` 大量为 0。

常见原因：

- 视场太窄（75°），手常出画
- 干等太久（`staged_static`：两腕速度 <0.015 m/s 且 ⩾1 s）
- 补帧 `filled` 被 QC 扔掉
- 双目一致性失败（重投影差、只一目见到）

你要做：看 `qc/yield.html` 拒绝原因；改拍摄（手进画面、少发呆）；100° 镜头；不要把失败帧用 0 填满。

8.5 坑 3：打不过「手永远不动」

现象：模型 ADE / val_loss ≈ `copy_current_wrist`；或更差。

人话：考试题太少时，「全程空着不动」的平均分也可能不差——因为很多帧本来就几乎静止。

仓库里的诚实现状（`docs/gap_analysis.md` / `PROGRESS.md`）：线性 BC 全量也就比保持不动好约 **6%**；小数据 ACT 常打平。

你要做：先加**同类任务**的段数；保证动作幅度真实；对比基线再谈改模型；别拿 pusht 仿真成功率当头戴人手成绩。

8.6 这一步你要做什么

- [ ] 每次录完先 `validate_session`，别直接训练
- [ ] 深度异常 → 查基线单位、同步、是否单目
- [ ] 产出率低 → 读 yield 报告原因，改拍摄再改阈值
- [ ] 训练不赢基线 → 先加数据，再改网络
- [ ] Day1 合格线表：`docs/hardware_day1_checklist.md` §4

---

9. 附录：命令速查与名词

9.1 想做 X，就跑 Y（均已在克隆中核对存在）

我想…	命令		
检查会话	`python3 scripts/validate_session.py /path/to/session`		
双目/HOT3D/iPhone 一条龙	`` `python3 scripts/run_stereo_pipeline.py --session\	--hot3d\	--iphone ... --out DIR`
Record3D → 会话	`python3 scripts/iphone/record3d_adapter.py take.r3d --out sessions/take`		
HOT3D → 假会话	`python3 scripts/simulate_device_session.py --hot3d CLIP --out DIR`		
头戴单条转 JSON	`python3 scripts/convert_headcam.py --session DIR --out episode.json`		
质检	`python3 scripts/egodata_qc.py --episodes DIR --html yield.html --csv yield.csv`		
导出 LeRobot	`python3 scripts/ego_to_lerobot.py --episodes DIR --yield-csv yield.csv --out LDIR --horizon 16`		
线性 BC	`python3 scripts/ego_pretrain_bc.py --dataset LDIR --max-frames 1000 --log train.log`		
可视化	`python3 scripts/ego_visualize.py overlay --episode E.json --out out.png --width 640`		
四级标注校验	`python3 scripts/validate_hierarchy.py examples/hierarchy_annotation.json --strict`		

9.2 名词（立刻解释）

词	意思
WiLoR	看图估计手部 3D 的模型（默认后端）
MANO	标准 3D 手模；权重需自行下载
三角化	用左右视差算深度
LiDAR	激光测距（iPhone Pro）
QC	自动质检
LeRobot	统一训练数据格式（parquet+mp4）
BC	行为克隆：模仿示范动作
ACT	一次预测一串动作的 Transformer 策略
ADE	轨迹平均位移误差（厘米）
hand_valid	这只手这帧能否计入损失
episode	一段连续录像 / 一条任务
零动作	永远预测「别动」的基线

9.3 建议阅读顺序

1. 本文第 0–1 章（采什么）
2. `docs/iphone_capture.md` 或 `docs/hardware_day1_checklist.md`
3. `docs/PIPELINE_ARCHITECTURE.md`
4. `docs/PROGRESS.md`（数字与待办）
5. `docs/headcam_data_spec.md` §7.7 / §8（格式权威）

命令以各脚本 `--help` 为准。若日后参数变更，以仓库代码为准。